

Aula 8 — Blazor Server

Módulo: Blazor

Carga horária: 4 horas

Projeto base: Nova aplicação **Blazor Server** integrada ao backend existente

Objetivo da aula:

Compreender a arquitetura do **Blazor Server**, seu funcionamento com **SignalR**, dominar o **ciclo de vida dos componentes**, **gerenciamento de estado**, **injeção de dependência** e implementar um **CRUD funcional com componentes reutilizáveis e comunicação entre componentes**.

PARTE 1 — TEORIA (≈ 2 horas)

1 Arquitetura do Blazor Server (SignalR)

◆ Como funciona?

- UI renderizada no **servidor**
- DOM atualizado via **SignalR**
- Navegador recebe apenas diffs de UI

Fluxo:

Browser ⇌ SignalR ⇌ Servidor (.NET)

◆ Características

- ✓ Código C# full-stack
 - ✓ Acesso direto a serviços e EF Core
 - ✓ Ideal para sistemas administrativos
 - ✗ Depende de conexão ativa
-

2 Ciclo de Vida dos Componentes

Principais métodos:

```
OnInitialized() OnInitializedAsync() OnParametersSet() OnAfterRender() OnAfterRenderAsync() Dispose()
```

Exemplo:

```
protected override async Task OnInitializedAsync() { await CarregarDadosAsync(); }
```

👉 Boas práticas:

- Dados → `OnInitializedAsync`
 - Parâmetros → `OnParametersSet`
 - JSInterop → `OnAfterRender`
-

3 State Management no Blazor Server

◆ Opções

- Estado local do componente
- Cascading Parameters
- Serviços Singleton/Scoped
- Session (com cuidado)

Exemplo simples:

```
public int Contador { get; set; }
```

Exemplo com serviço:

```
builder.Services.AddScoped<AppState>();
```

Dependency Injection no Blazor

Blazor usa a **DI** nativa do ASP.NET Core.

```
@inject ProdutoService ProdutoService
```

Ciclos recomendados:

Serviço	Ciclo
State	Scoped
Dados	Scoped
Cache	Singleton

PARTE 2 — PRÁTICA (≈ 2 horas)

Objetivo Prático

Criar um **CRUD** completo de Produtos em Blazor Server, com:

- Componentes reutilizáveis
- Comunicação entre componentes
- State compartilhado

Criando o Projeto Blazor Server

```
dotnet new blazorserver -n Aula08.BlazorServer cd Aula08.BlazorServer dotnet run
```

2 Estrutura do Projeto

```
Aula08.BlazorServer |— Pages | |— Produtos.razor |— Components | |— ProdutoForm.razor | |— ProdutoList.razor |— Services |
|— ProdutoService.cs |— Models | |— Produto.cs
```

3 Model Produto

Models/Produto.cs

```
namespace Aula08.BlazorServer.Models { public class Produto { public int Id { get; set; } public string Nome { get; set; } public
decimal Preco { get; set; } } }
```

4 Serviço de Dados (State + CRUD)

Services/ProdutoService.cs

```
using Aula08.BlazorServer.Models; namespace Aula08.BlazorServer.Services { public class ProdutoService { private List<Produto>
_produtos = new(); private int _id = 1; public List<Produto> ObterTodos() => _produtos; public void Adicionar(Produto produto) {
produto.Id = _id++; _produtos.Add(produto); } public void Remover(int id) { var produto = _produtos.First(p => p.Id == id);
_produtos.Remove(produto); } } }
```

📌 Registrar no Program.cs :

```
builder.Services.AddScoped<ProdutoService>();
```

5 Página Principal (CRUD)

 Pages/Produtos.razor

```
@page "/produtos" @using Aula08.BlazorServer.Models @inject ProdutoService ProdutoService <h3>Produtos</h3> <ProdutoForm
OnSalvar="AdicionarProduto" /> <ProdutoList Produtos="Produtos" OnExcluir="ExcluirProduto" /> @code { private List<Produto>
Produtos = new(); protected override void OnInitialized() { Produtos = ProdutoService.ObterTodos(); } void
AdicionarProduto(Produto produto) { ProdutoService.Adicionar(produto); StateHasChanged(); } void ExcluirProduto(int id) {
ProdutoService.Remover(id); StateHasChanged(); } }
```

6 Componente de Formulário Reutilizável

 Components/ProdutoForm.razor

```
@using Aula08.BlazorServer.Models <EditForm Model="Produto" OnValidSubmit="Salvar"> <InputText @bind-Value="Produto.Nome"
placeholder="Nome" /> <InputNumber @bind-Value="Produto.Preco" placeholder="Preço" /> <button type="submit">Salvar</button>
</EditForm> @code { private Produto Produto = new(); [Parameter] public EventCallback<Produto> OnSalvar { get; set; } async Task
Salvar() { await OnSalvar.InvokeAsync(Produto); Produto = new Produto(); // reset } }
```

7 Componente de Lista (Comunicação Pai ⇄ Filho)

 Components/ProdutoList.razor

```
@using Aula08.BlazorServer.Models <table> @foreach (var p in Produtos) { <tr> <td>@p.Nome</td> <td>@p.Preco</td> <td> <button
@onclick="() => OnExcluir.InvokeAsync(p.Id)"> Excluir </button> </td> </tr> } </table> @code { [Parameter] public List<Produto>
Produtos { get; set; } [Parameter] public EventCallback<int> OnExcluir { get; set; } }
```

- ✓ Comunicação via EventCallback
- ✓ Reutilizável
- ✓ Baixo acoplamento

8 Ciclo de Vida em Ação

```
protected override void OnInitialized() { // Carrega dados iniciais }
```

```
StateHasChanged(); // força renderização
```

✓ Resultado Esperado

- ✓ CRUD funcional em Blazor Server
 - ✓ Componentes reutilizáveis
 - ✓ Comunicação entre componentes
 - ✓ State compartilhado
 - ✓ Base sólida para Blazor avançado
-

📌 Atividade Avaliativa

👉 Desafio prático:

1. Criar componente `ProdutoDetalhe`
2. Usar `CascadingParameter`
3. Implementar confirmação de exclusão
4. Persistir estado em serviço